

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по лабораторной работе №2
“Объектно-ориентированные возможности языка Python”

Выполнил:
Студент группы ИУ5-35Б
Иванов Б. М.
Преподаватель:
Гапанюк Ю. Е.

Москва 2025

Цель лабораторной работы

Цель лабораторной работы: изучение объектно-ориентированных возможностей языка Python.

Задание лабораторной работы

1. Необходимо создать виртуальное окружение и установить в него хотя бы один внешний пакет с использованием `pip`.
2. Необходимо разработать программу, реализующую работу с классами. Программа должна быть разработана в виде консольного приложения на языке Python 3.
3. Все файлы проекта (кроме основного файла `main.py`) должны располагаться в пакете `lab_python_oop`.
4. Каждый из нижеперечисленных классов должен располагаться в отдельном файле пакета `lab_python_oop`.
5. Абстрактный класс «Геометрическая фигура» содержит абстрактный метод для вычисления площади фигуры.
6. Класс «Цвет фигуры» содержит свойство для описания цвета геометрической фигуры.
7. Класс «Прямоугольник» наследуется от класса «Геометрическая фигура». Класс должен содержать конструктор по параметрам «ширина», «высота» и «цвет». В конструкторе создается объект класса «Цвет фигуры» для хранения цвета. Класс должен переопределять метод, вычисляющий площадь фигуры.
8. Класс «Круг» создается аналогично классу «Прямоугольник», задается параметр «радиус». Для вычисления площади используется константа `math.pi` из модуля `math`.
9. Класс «Квадрат» наследуется от класса «Прямоугольник». Класс должен содержать конструктор по длине стороны. Для классов «Прямоугольник», «Квадрат», «Круг»:
 - Определите метод `"repr"`, который возвращает в виде строки основные параметры фигуры, ее цвет и площадь. Используйте метод `format`.
 - Название фигуры («Прямоугольник», «Квадрат», «Круг») должно задаваться в виде поля данных класса и возвращаться методом класса.

10. В корневом каталоге проекта создайте файл `main.py` для тестирования Ваших классов. Создайте следующие объекты и выведите о них информацию в консоль (N - номер Вашего варианта по списку группы):

- Прямоугольник синего цвета шириной N и высотой N.
- Круг зеленого цвета радиусом N.
- Квадрат красного цвета со стороной N.
- Также вызовите один из методов внешнего пакета, установленного с использованием `pip`.

11. **Дополнительное задание.** Протестируйте корректность работы Вашей программы с помощью модульного теста.

Текст программы

Файл GeometricalShape.py

```
from abc import ABC, abstractmethod
class GeometricalShape(ABC):
    @abstractmethod
    def area(self)->float:
        pass
    @abstractmethod
    def get_name(self)->str:
        pass
    @abstractmethod
    def draw(self, ax)->None:
        pass
    @abstractmethod
    def __repr__(self):
        pass
```

Файл Circle.py

```
from lab2.lab_python_oop import GeometricalShape, Shape_Color
import math
from matplotlib.patches import Circle as MplCircle
class Circle(GeometricalShape):
    def __init__(self, radius:float, color:Shape_Color.Color):
        self._name = "Круг"
        if radius <= 0.0:
            raise ValueError("Радиус должен быть положительным числом")
        self._radius = radius
        self._color = color
    def get_name(self)->str:
        return self._name
    @property
    def color(self)->str:
        return str(self._color)
    @color.setter
    def color(self, color_name:Shape_Color.Color):
        self._color=color_name
    @property
    def radius(self)->float:
        return self._radius
    @radius.setter
    def radius(self, radius:float):
        if radius <= 0.0:
            raise ValueError("Радиус должен быть положительным числом")
        self._radius=radius
    def area(self)->float:
        return math.pi * self._radius ** 2
    def draw(self, ax):
        circle = MplCircle((0,0), self._radius, fill=True, facecolor=str(self._color),
                            edgecolor='red', linewidth=2, label=f'Круг R={self._radius}')
        ax.add_patch(circle)
    def __repr__(self):
        return f"{self._name} с радиусом {self._radius} цвета {str(self._color)}. Площадь: {self.area():.2f}"
```

Файл Rectangle.py

```
from lab2.lab_python_oop import GeometricalShape, Shape_Color
from matplotlib.patches import Rectangle as MplRectangle
class Rectangle(GeometricalShape):
    def __init__(self, width:float, height:float, color:Shape_Color.Color):
        self._name="Прямоугольник"
        if width <= 0 or height <= 0:
            raise ValueError("Стороны должны быть положительными числами")
        self._width = width
        self._height = height
        self._center = (0, 0)
        self._color = color
    def get_name(self)->str:
        return self._name
    @property
    def color(self)->str:
        return str(self._color)
    @color.setter
    def color(self, color_name:Shape_Color.Color):
        self._color=color_name
    @property
    def width(self)->float:
        return self._width
    @width.setter
    def width(self, width:float):
        self._width=width
    @property
    def height(self)->float:
        return self._height
    @height.setter
    def height(self, height:float):
        self._height=height
    def area(self):
        return self._width * self._height
    def draw(self, ax):
        rectangle = MplRectangle((-self._width/2.0,-self._height/2.0), width=self._width,height=self._height, fill=True,
        facecolor=str(self._color),
        edgecolor='red', linewidth=2, label=f'{self._name}')
        ax.add_patch(rectangle)
    def __repr__(self):
        return f'{self._name} со сторонами {self._width} и {self._height} цвета {str(self._color)}. Площадь: {self.area():.2f}'
```

Файл Shape Color.py

```
from enum import Enum
class Color(Enum):
    BLUE = "blue"
    RED = "red"
    GREEN = "green"
    def __str__(self):
        return self.value
```

Файл Square.py

```
from lab2.lab_python_oop.Rectangle import Rectangle, Shape_Color
class Square(Rectangle):
    def __init__(self, side: float, color:Shape_Color.Color):
        super().__init__(side, side, color)
        self._name = "Квадрат"
    def __repr__(self):
        return f'{self._name} со стороной {self.width} цвета {str(self.color)}. Площадь: {self.area():.2f}'
```

Файл main.py

```
from lab2.lab_python_oop import Circle, Rectangle, Square, Color
import matplotlib.pyplot as plt
def main():
    n = 9.0
    circle = Circle(radius=n, color=Color.GREEN)
    rectangle = Rectangle(width=n, height=n,color=Color.BLUE)
    square = Square(side=n,color=Color.RED)
    shapes = [circle,rectangle,square]
    for shape in shapes:
        fig, ax = plt.subplots(figsize=(10, 8))
        shape.draw(ax)
        print(shape)
        ax.set_aspect('equal')
        ax.set_xlim(-(n + 2), n + 2)
        ax.set_ylim(-(n + 2), n + 2)
        ax.set_title(shape.get_name())
        plt.savefig(f'./figures/{shape.get_name()}.png')
        print(f'Изображение сохранено в '{shape.get_name()}.png'')
        print("-" * 40)
if __name__ == "__main__":
    main()
```

Тесты к программе

Файл test_circle.py

```
import unittest
from lab2.lab_python_oop import Circle, Color
import matplotlib.pyplot as plt
class TestCircle(unittest.TestCase):
    def setUp(self):
        self.circle = Circle(radius=5.0, color=Color.GREEN)
    def test_circle_creation(self):
        self.assertEqual(self.circle.radius, 5.0)
        self.assertEqual(str(self.circle.color), "green")
    def test_circle_area(self):
        expected_area = 3.141592653589793 * 25
        self.assertAlmostEqual(self.circle.area(), expected_area, places=2)
    def test_circle_negative_radius(self):
        with self.assertRaises(ValueError):
            Circle(radius=-5.0, color=Color.GREEN)
    def test_circle_draw(self):
        fig, ax = plt.subplots()
        try:
            self.circle.draw(ax)
            self.assertTrue(True)
        except Exception as e:
            self.fail(f"Ошибка при отрисовке: {e}")
        finally:
            plt.close(fig)
    def test_circle_repr(self):
        expected_repr = "Круг с радиусом 5.0 цвета green. Площадь: 78.54"
        self.assertEqual(repr(self.circle), expected_repr)
if __name__ == '__main__':
    unittest.main()
```

Файл test_rectangle.py

```
import unittest
from lab2.lab_python_oop import Rectangle, Color
import matplotlib.pyplot as plt
class TestRectangle(unittest.TestCase):
    def setUp(self):
        self.rectangle = Rectangle(width=6.0, height=4.0, color=Color.RED)
    def test_rectangle_creation(self):
        self.assertEqual(self.rectangle.width, 6.0)
        self.assertEqual(self.rectangle.height, 4.0)
        self.assertEqual(str(self.rectangle.color), "red")
    def test_rectangle_area(self):
        self.assertEqual(self.rectangle.area(), 24.0)
    def test_rectangle_negative_sides(self):
        with self.assertRaises(ValueError):
            Rectangle(width=-6.0, height=4.0, color=Color.RED)
        with self.assertRaises(ValueError):
            Rectangle(width=6.0, height=-4.0, color=Color.RED)
    def test_rectangle_draw(self):
        fig, ax = plt.subplots()
        try:
            self.rectangle.draw(ax)
            self.assertTrue(True)
        except Exception as e:
            self.fail(f"Ошибка при отрисовке: {e}")
        finally:
            plt.close(fig)
    def test_rectangle_repr(self):
        expected_repr = "Прямоугольник со сторонами 6.0 и 4.0 цвета red. Площадь: 24.00"
        self.assertEqual(repr(self.rectangle), expected_repr)
if __name__ == '__main__':
    unittest.main()
```

Файл test_square.py

```
import unittest
from lab2.lab_python_oop import Square, Color
import matplotlib.pyplot as plt
class TestSquare(unittest.TestCase):
    def setUp(self):
        self.square = Square(side=5.0, color=Color.BLUE)
    def test_square_creation(self):
        self.assertEqual(self.square.width, 5.0)
        self.assertEqual(self.square.height, 5.0)
        self.assertEqual(str(self.square.color), "blue")
    def test_square_area(self):
        self.assertEqual(self.square.area(), 25.0)
    def test_square_negative_side(self):
        with self.assertRaises(ValueError):
            Square(side=-5.0, color=Color.BLUE)
    def test_square_draw(self):
        fig, ax = plt.subplots()
        try:
            self.square.draw(ax)
            self.assertTrue(True)
        except Exception as e:
            self.fail(f"Ошибка при отрисовке: {e}")
        finally:
            plt.close(fig)
    def test_square_repr(self):
        expected_repr = "Квадрат со стороной 5.0 цвета blue. Площадь: 25.00"
        self.assertEqual(repr(self.square), expected_repr)
if __name__ == '__main__':
    unittest.main()
```

Файл test_main.py

```
import unittest
import os
import tempfile
import shutil
from unittest.mock import patch, MagicMock
from lab2.main import main
class TestMain(unittest.TestCase):
    def setUp(self):
        self.test_dir = tempfile.mkdtemp()
        self.original_cwd = os.getcwd()
        os.chdir(self.test_dir)
        os.makedirs('figures', exist_ok=True)
    def tearDown(self):
        os.chdir(self.original_cwd)
        shutil.rmtree(self.test_dir)
    def test_main_execution(self):
        try:
            main()
            self.assertTrue(os.path.exists('figures/Круг.png'))
            self.assertTrue(os.path.exists('figures/Прямоугольник.png'))
            self.assertTrue(os.path.exists('figures/Квадрат.png'))
        except Exception as e:
            self.fail(f'main() вызвал исключение: {e}')
    @patch('matplotlib.pyplot.savefig')
    @patch('matplotlib.pyplot.subplots')
    @patch('builtins.print')
    def test_main_mocked(self, mock_print, mock_subplots, mock_savefig):
        mock_ax = MagicMock()
        mock_fig = MagicMock()
        mock_subplots.return_value = (mock_fig, mock_ax)
        main()
        self.assertEqual(mock_savefig.call_count, 3)
        self.assertTrue(mock_print.called)
if __name__ == '__main__':
    unittest.main()
```

Файл run_tests.py

```
import unittest
import sys
import os
sys.path.insert(0, os.path.abspath('.'))
test_loader = unittest.TestLoader()
test_suite = test_loader.discover('tests', pattern='test_*.py')
test_runner = unittest.TextTestRunner(verbosity=2)
result = test_runner.run(test_suite)
sys.exit(not result.wasSuccessful())
```


Экранные формы с примерами выполнения программы

```
/home/boris/PycharmProjects/PythonProject/.venv/bin/python /home/boris/PycharmProjects/PythonProject/lab2/main.py
```

Круг с радиусом 9.0 цвета green. Площадь: 254.47

Изображение сохранено в 'Круг.png'

Прямоугольник со сторонами 9.0 и 9.0 цвета blue. Площадь: 81.00

Изображение сохранено в 'Прямоугольник.png'

Квадрат со стороной 9.0 цвета red. Площадь: 81.00

Изображение сохранено в 'Квадрат.png'

